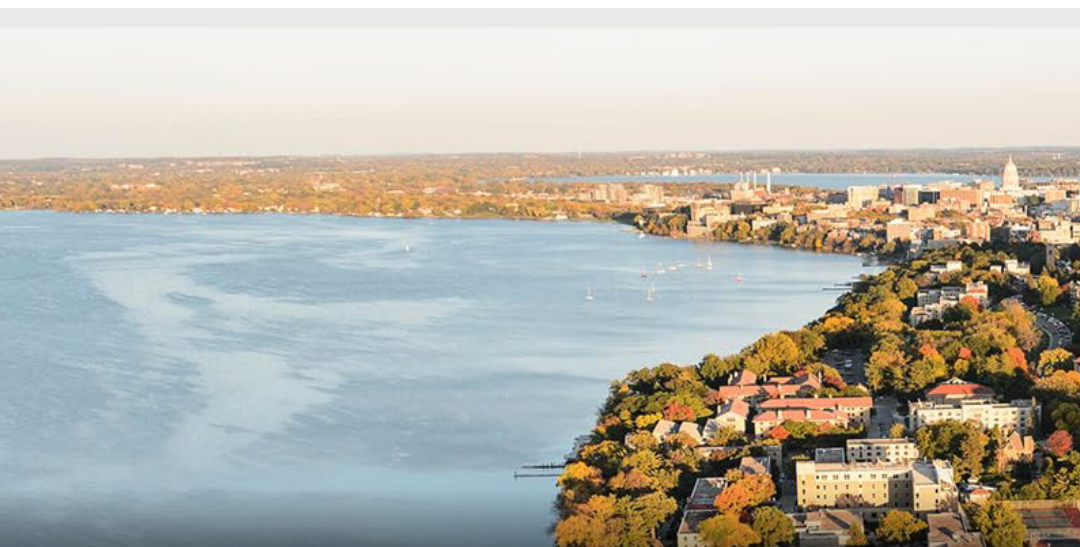




CS 540 Introduction to Art **Games I**

University of Wisconsin
Fall 2025 Section

Announcements

- **Homework:**
 - HW8 due on **Friday November 2**

- Class roadmap:

Games – Pa

Introduction
Learning

Reinforcement

Outline

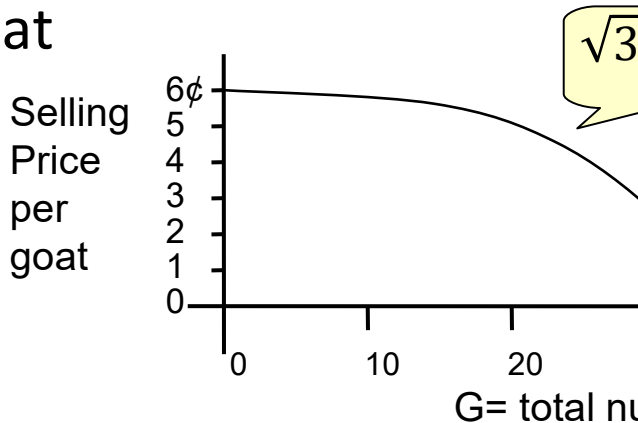
- Games I Review
- Sequential-move games
 - Game trees, minimax, search algorithms
- Speeding up sequential-move games
 - Pruning, heuristics

Games I Review

- Game Properties
- Simultaneous-Move Games
- Strictly Dominant Strategy and Dominant Strategy Equilibrium
- Nash Equilibrium
 - Pure Nash Equilibrium
 - Mixed Strategy Nash Equilibrium

Pure NE in an Infinite The tragedy of the

- Price per goat



- How many goats should one (out of A)
- How much would the farmer earn?

Continuous Act

- Each farmer has infinite number of
- The value for farmer i , when the n farmers choose $(g_1, \dots, g_n)$ is

$$u_i(g_1, g_2, \dots, g_n) = g_i$$

- **Assume** a pure Nash equilibrium exists
- **Assume** (by apparent symmetry) that

Finding

- $u_i(g_1, g_2, \dots, g_n) = g_i \sqrt{36 - (n-1)g_i^2}$
- g^* is the best response to other

$$g^* = \operatorname{argmax}_{h \in [0, 36]} u_i(g^*, \dots, g^*, h)$$

$$= \operatorname{argmax}_h h \sqrt{36 - (n-1)h^2}$$

Finding g^*

$$g^* = \operatorname{argmax}_h h \sqrt{36 - (n-1)h}$$

- Taking derivative w.r.t. h of g

$$g^* = \frac{72 - 2(n-1)}{3}$$

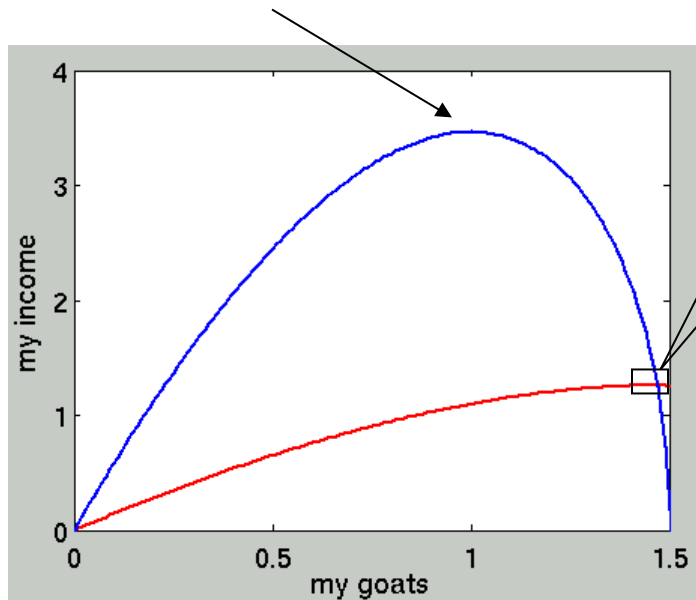
$$g^* = \frac{72}{2n+1}$$

The tragedy of the C

- Say there are $n=24$ farmers.
Each would **rationally** graze $g_i^* =$
- Each would get **1.25¢**
- But if they cooperate and each g
- Each would get **3.46¢**

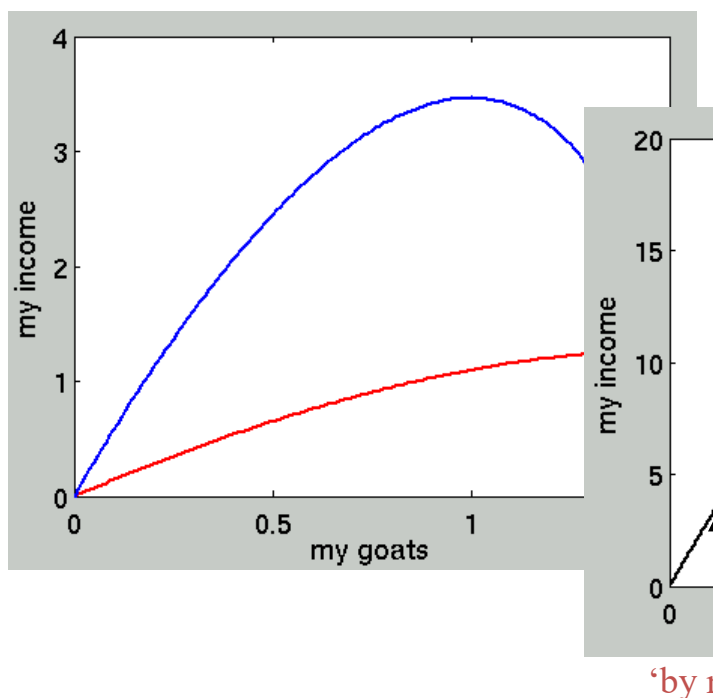
The tragedy of the

If all 24 farmers agree on the same number of goats to raise, 1 goat per farmer would be optimal



The tragedy of the

If all 24 farmers agree on the same number of goats to raise, 1 goat per farmer would be optimal



The tragedy

- Rational behaviors lead to sub-optimal outcomes
- Maximizing individual welfare not necessarily maximizes social welfare
- What went wrong?

Shouldn't have allowed **free** grazing?

It's not just the goats: Common problem

Mechanism design: designing the rules

Outline

- Games I Review
- Sequential-move games
 - Game trees, minimax, search algorithms
- Speeding up sequential-move games
 - Pruning, heuristics

Sequential-Mov

More complex games with mult

- Instead of normal form, **extensive**
 - Represent with a **tree**
 - **Rewards at leaves**
 - Find strategies: perform search over
-
- Nash equilibrium still well-def
 - Backward induction

II-Nim: Example Sequen

2 piles of sticks, each with 2 sticks

- Each player takes one or more sticks
- Take last stick: lose (ii, ii)

- Two players: **Max** and **Min**
- If **Max** wins, its score is **+1**; otherwise
- **Min**'s score is **-Max's** (two-player zero-sum)
- Use **Max's** as the score of the game

Game Traje
(ii, ii)

Game Traje

(ii, ii)

Max takes one stick from

(i, ii)

Game Traje

(ii, ii)

Max takes one stick from

(i, ii)

Min takes two sticks from

(i, -)

Game Traje

(ii, ii)

Max takes one stick from

(i, ii)

Min takes two sticks from

(i, -)

Max takes the last

(-, -)

Max gets score

Game tree for

Two players:
Max and **Min**

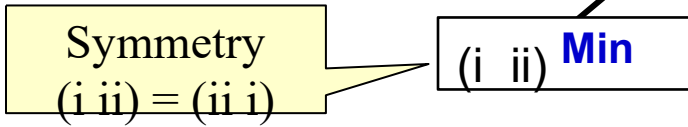
(ii ii) **Max**



Convention: score is w.r.t. the first player Max. Min's score = $- \text{Max}$

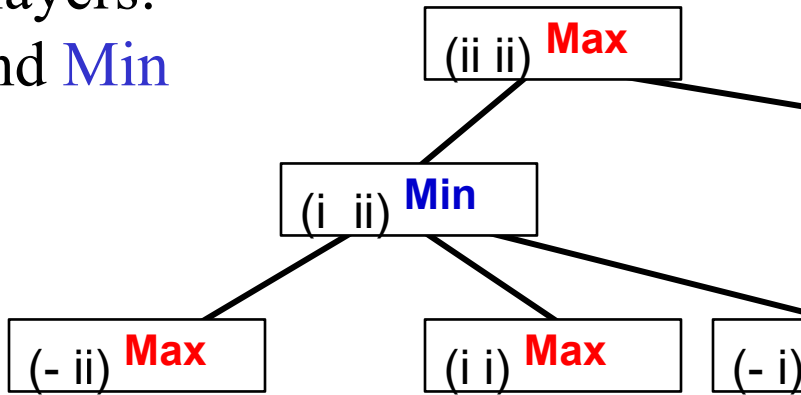
Game tree for

Two players:
Max and **Min**



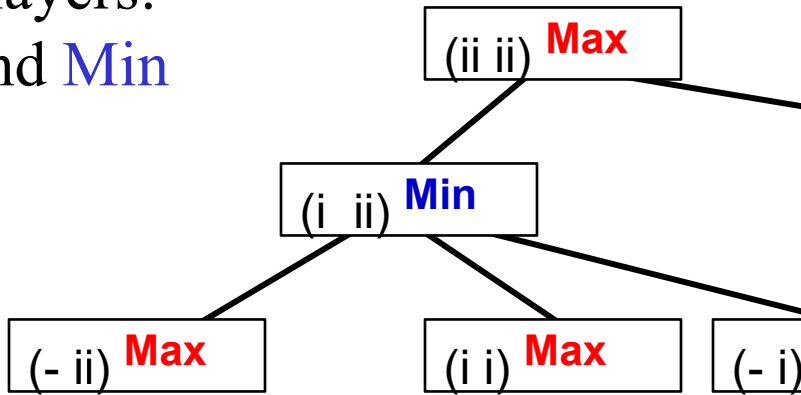
Game tree for

Two players:
Max and **Min**



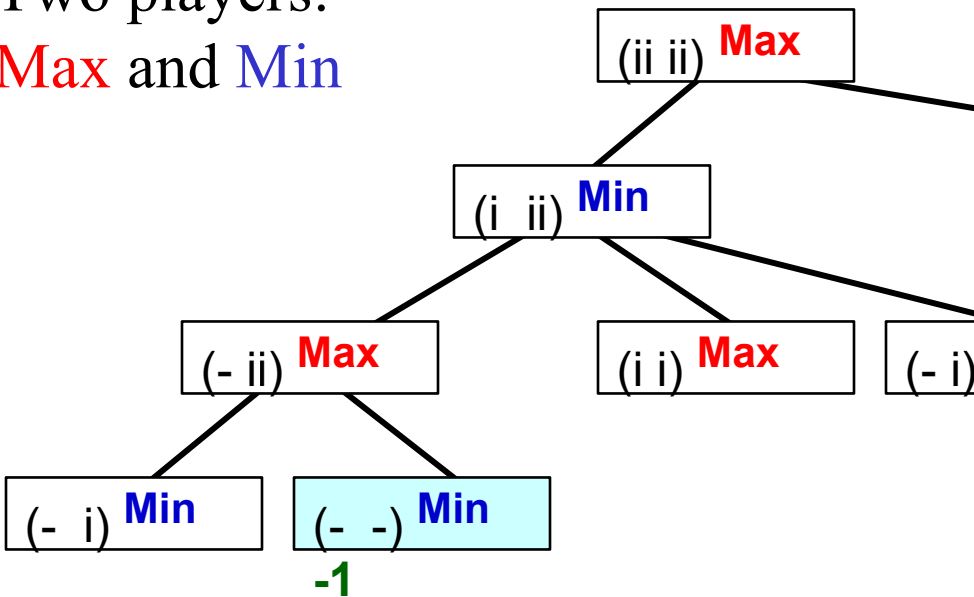
Game tree for

Two players:
Max and **Min**



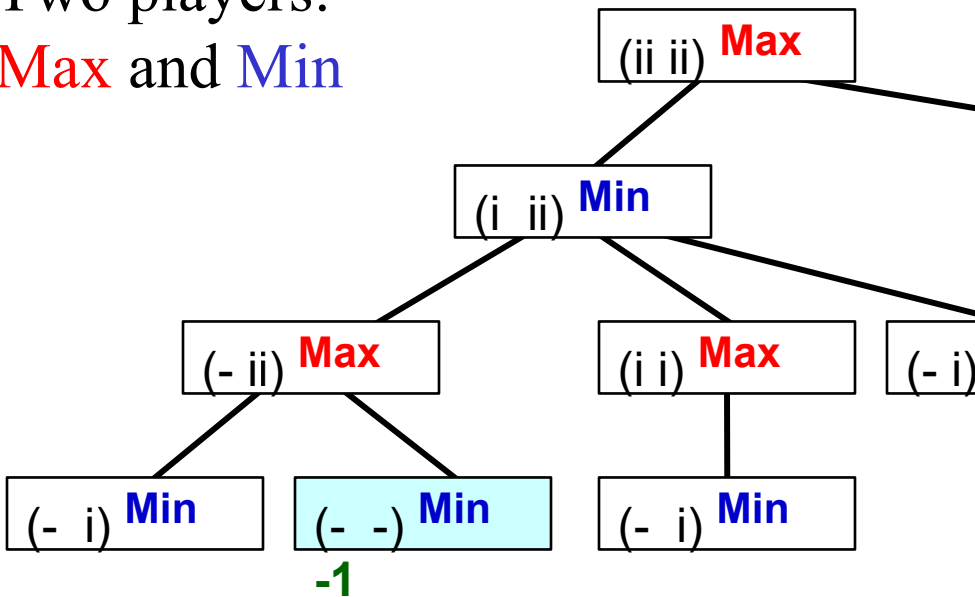
Game tree for

Two players:
Max and **Min**



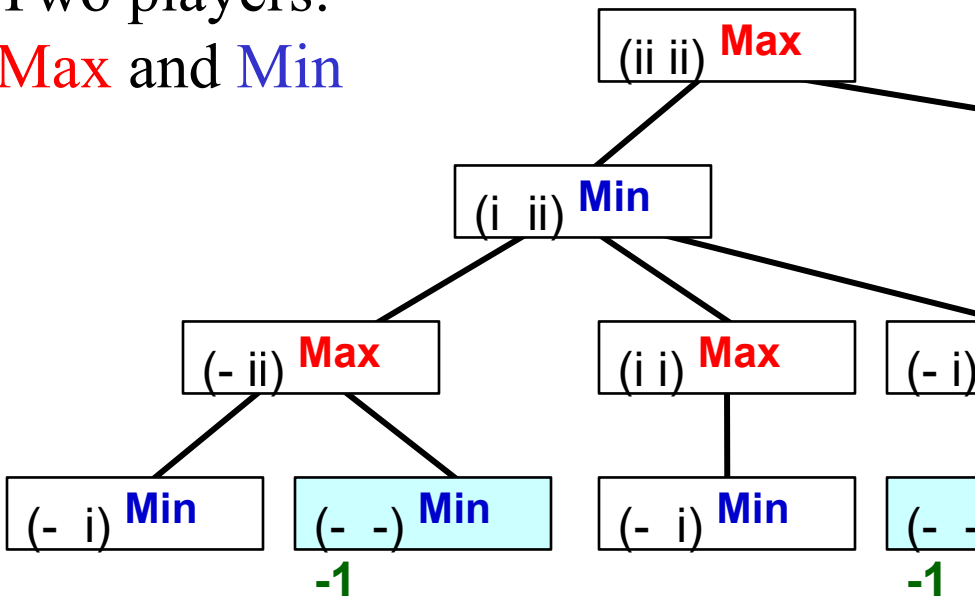
Game tree for

Two players:
Max and **Min**



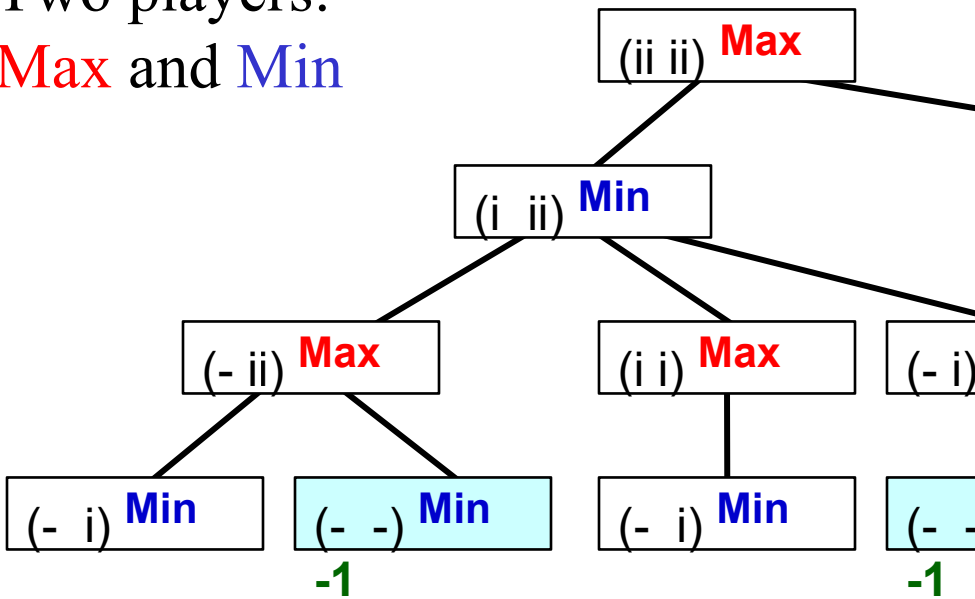
Game tree for

Two players:
Max and **Min**



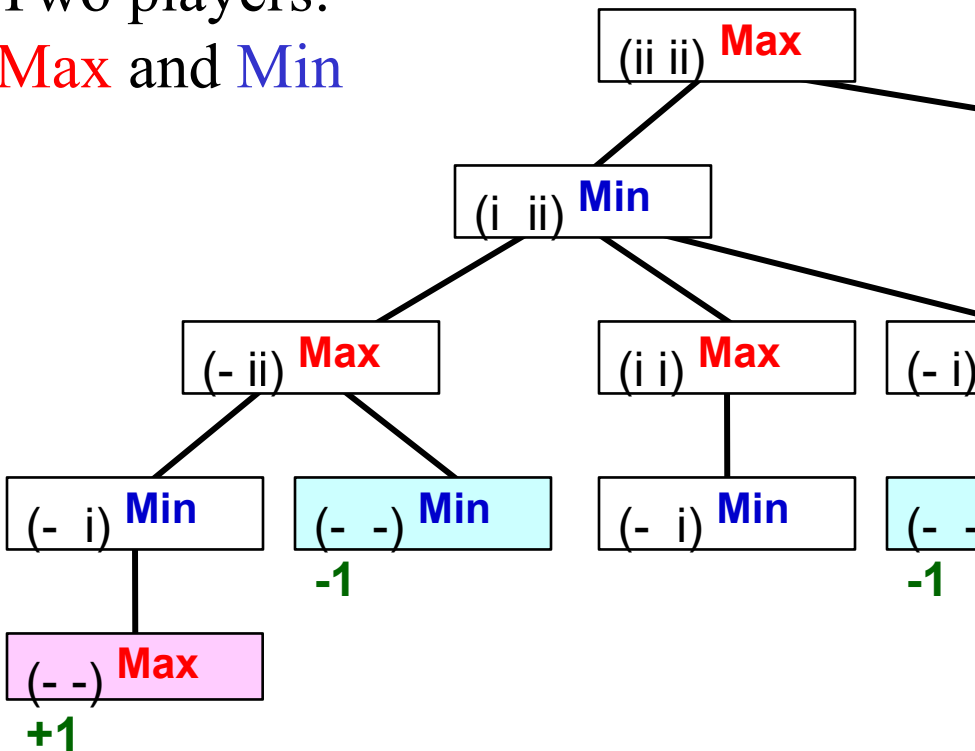
Game tree for

Two players:
Max and **Min**



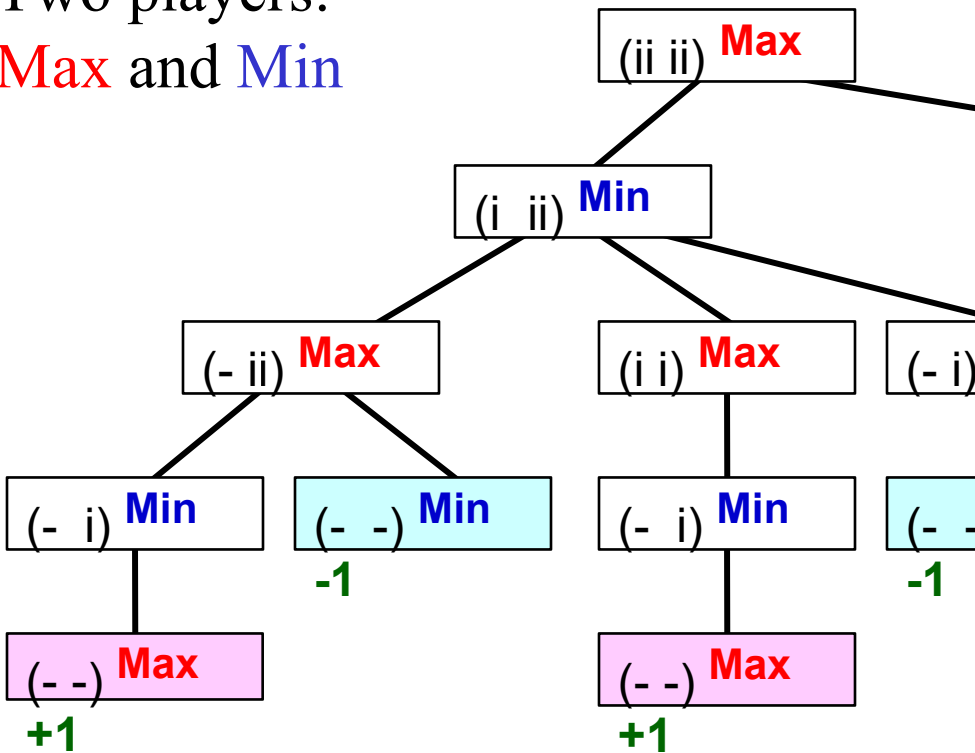
Game tree for

Two players:
Max and **Min**



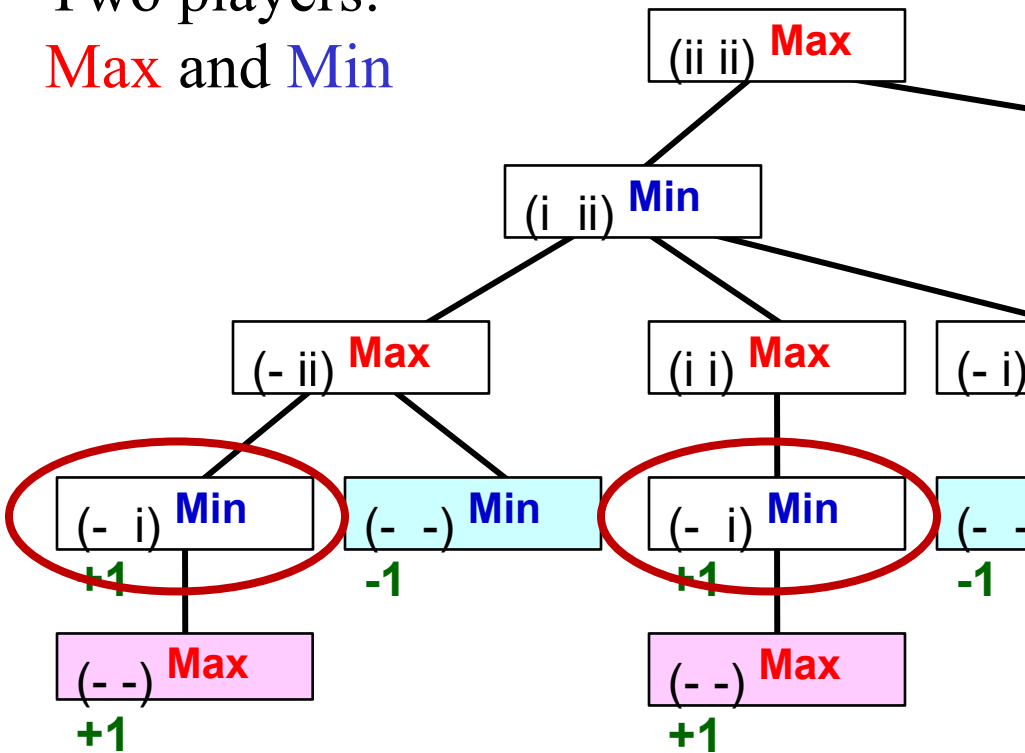
Game tree for

Two players:
Max and **Min**



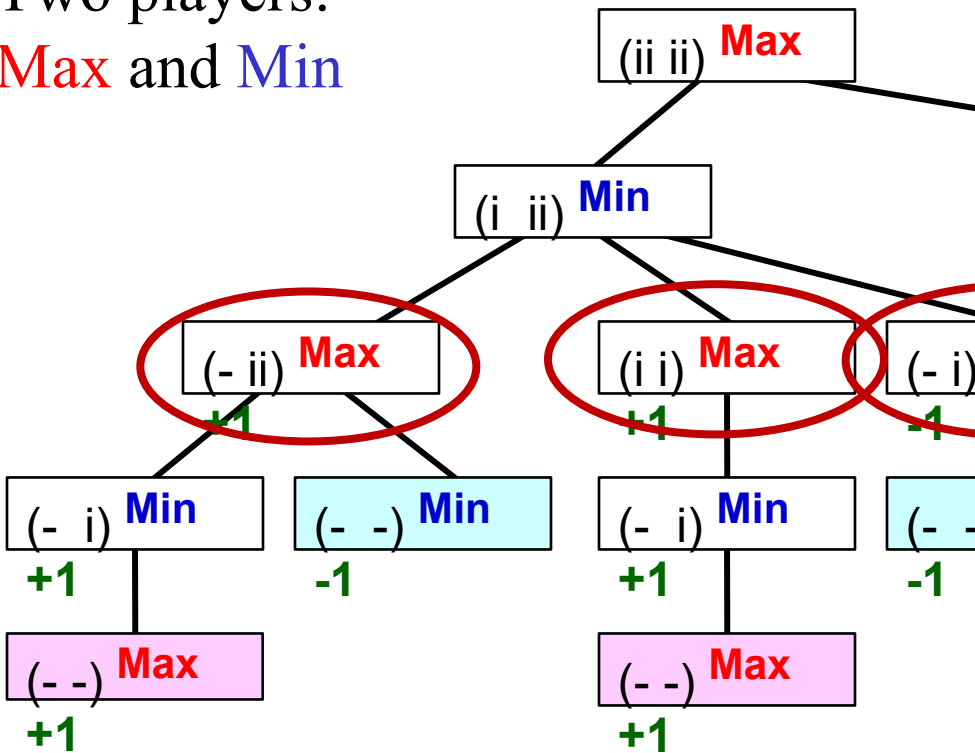
Game tree for

Two players:
Max and **Min**



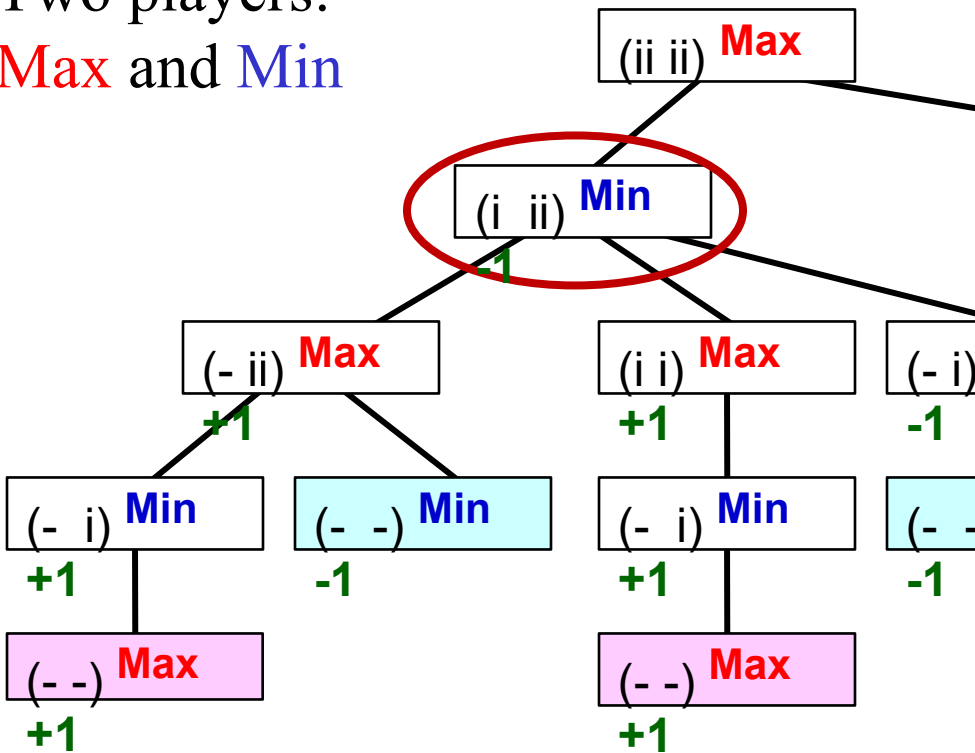
Game tree for

Two players:
Max and **Min**



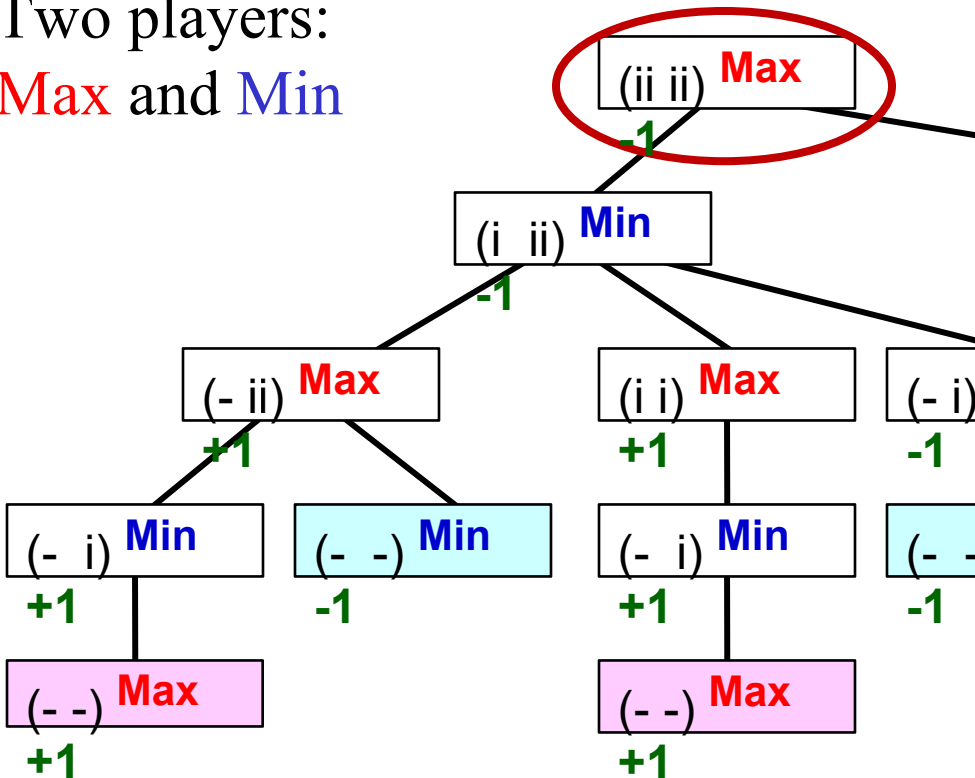
Game tree for

Two players:
Max and **Min**



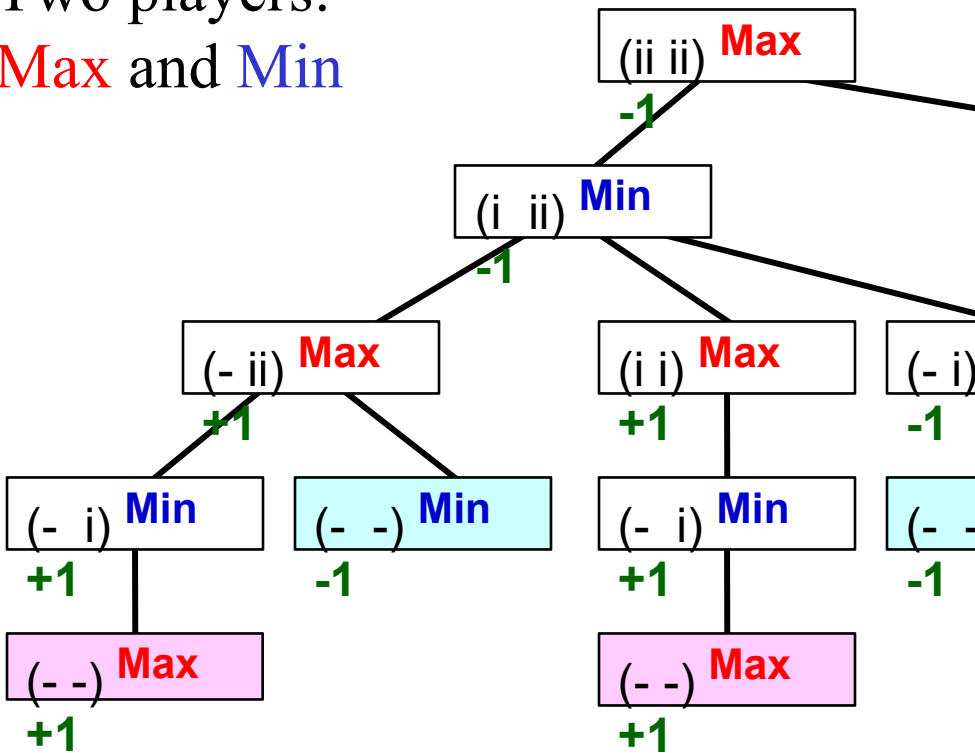
Game tree for

Two players:
Max and **Min**



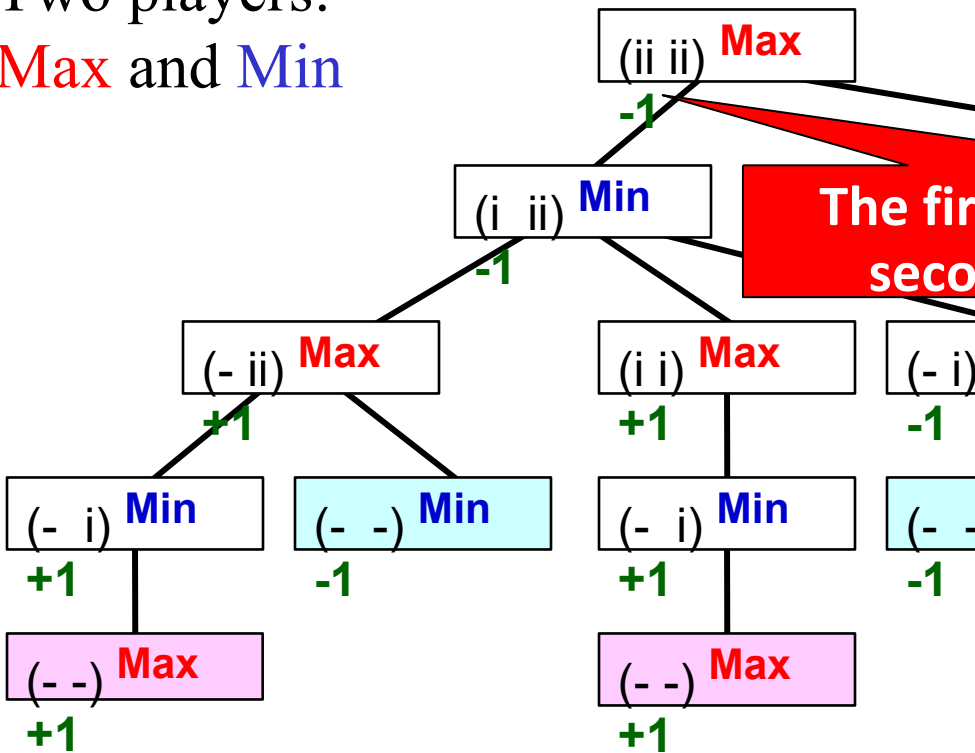
Game tree for

Two players:
Max and **Min**



Game tree for

Two players:
Max and **Min**



Break & C

Q 2.1: We are playing a game where Player A goes first and has 3 moves. Player B goes next and has 3 moves. Player A then has 3 moves. Player B then has one move.

How many nodes are there in the minimax tree (leaves)?

- A. 23
- B. 65
- C. 41
- D. 2

Break & C

Q 2.1: We are playing a game where Player A goes first and has 3 moves. Player B goes next and has 3 moves. Player A then has 3 moves. Player B then has one move.

How many nodes are there in the minimax tree (leaves)?

- A. 23
- **B. 65**
- C. 41
- D. 2

Break & C

Q 2.1: We are playing a game where Player A starts and has 3 moves. Player B goes next and has 3 moves. Player A then has 3 moves. Player B then has one move.

How many nodes are there in the minimax tree (leaves)?

- A. 23
- **B. 65** ($1 + 4 + 4*3 + 4*3*2 + 4*3*2 = 65$ nodes.)
- C. 41
- D. 2

Our Approach

We find the minimax value/strategy

- Minimax value: score of terminal node if both players play optimally
 - **Max's** turn, take max of children
 - **Min's** turn, take min of children
- Can implement this as depth-first search

Minimax Algo

function **Max-Value**(s)

inputs:

s: current state in game, Max about to play

output: *best-score (for Max) available from s*

if (s is a terminal state)

then return (terminal value of s)

else

$\alpha := -\text{infinity}$

for each s' in Succ(s)

$\alpha := \max(\alpha, \text{Min-value}(s'))$

return α

function **Min-Value**(s)

output: *best-score (for Min) available from s*

if (s is a terminal state)

then return (terminal value of s)

else

$\beta := \text{infinity}$

for each s' in Succs(s)

$\beta := \min(\beta, \text{Max-value}(s'))$

return β

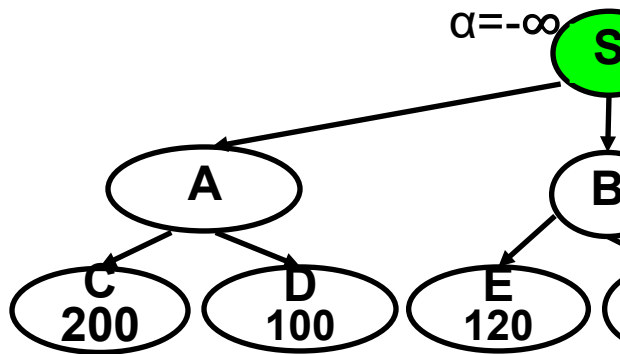
Minimax algorithm

max

min

max

min



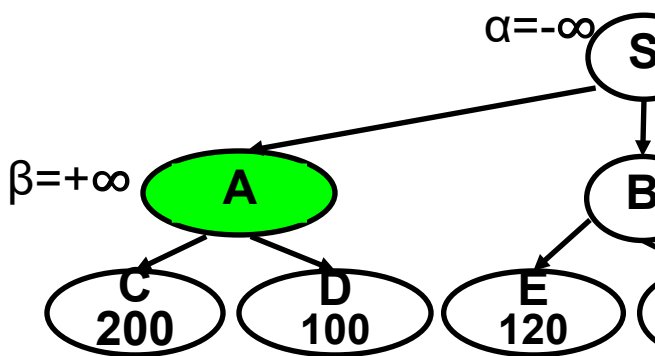
Minimax algorithm

max

min

max

min



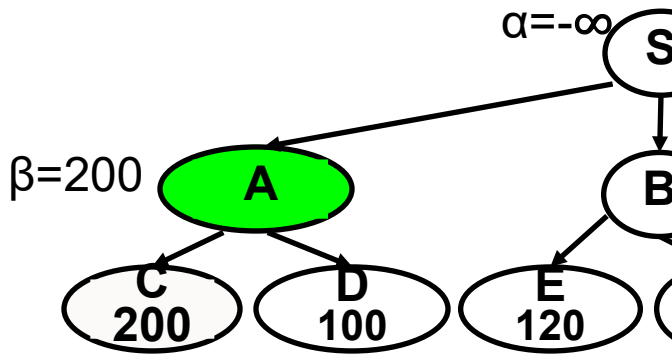
Minimax algorithm

max

min

max

min



The execution on the terminal nodes is omitted.

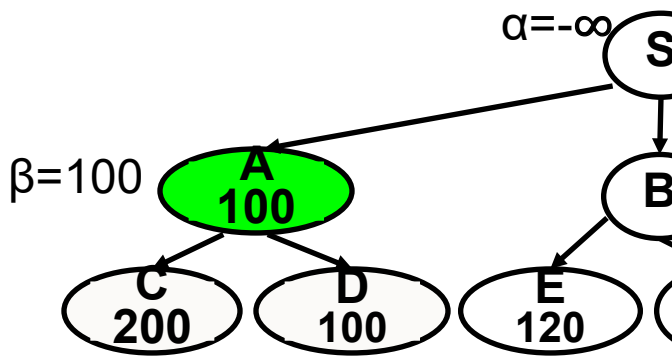
Minimax algorithm

max

min

max

min



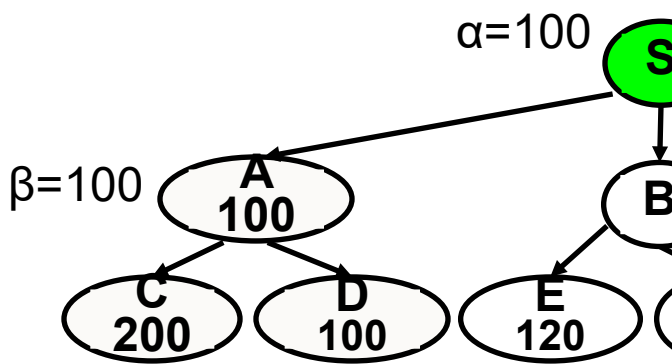
Minimax algorithm

max

min

max

min



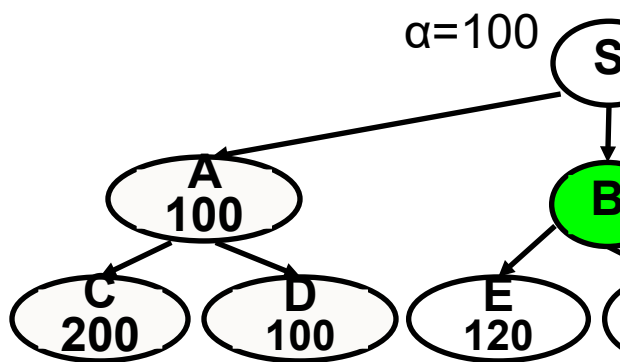
Minimax algorithm

max

min

max

min



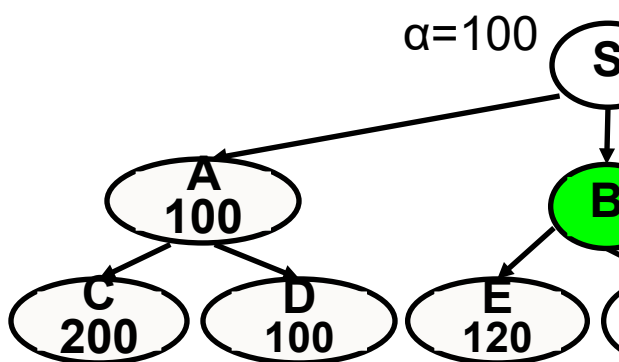
Minimax algorithm

max

min

max

min



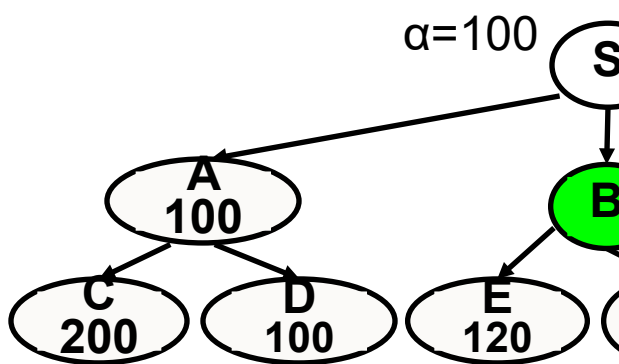
Minimax algorithm

max

min

max

min



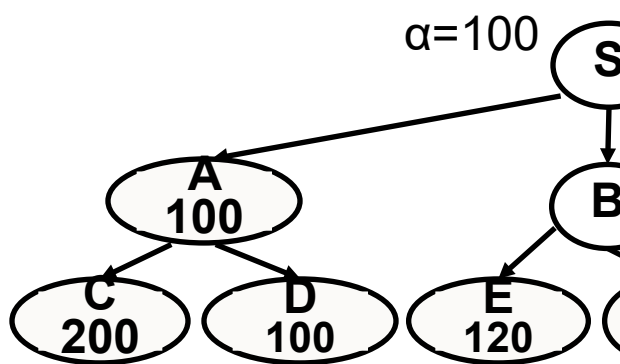
Minimax algorithm

max

min

max

min



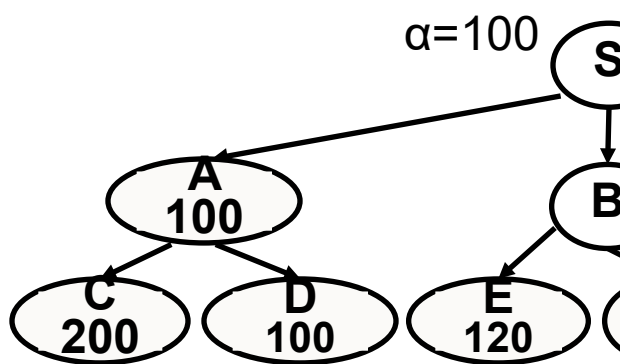
Minimax algorithm

max

min

max

min



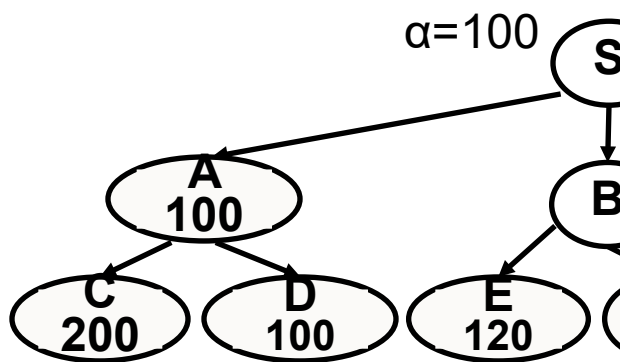
Minimax algorithm

max

min

max

min



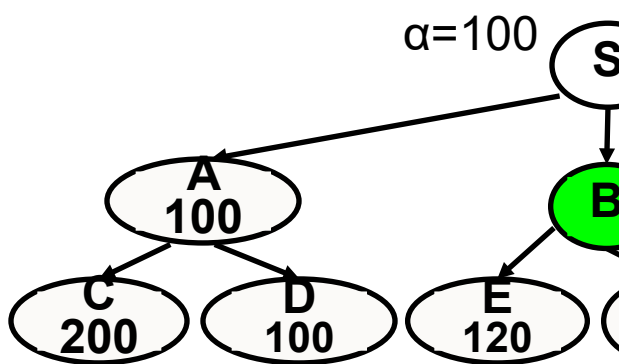
Minimax algorithm

max

min

max

min



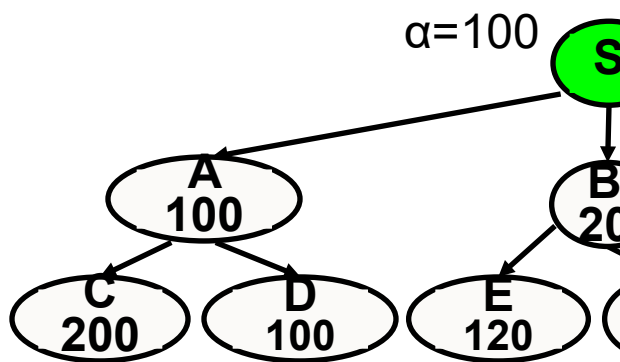
Minimax algorithm

max

min

max

min



Break & C

Q 2.2: During minimax tree search, must

- A. Always
- B. Sometimes
- C. Never

Break & C

Q 2.2: During minimax tree search, must

- A. Always
- **B. Sometimes**
- C. Never

Break & C

Q 2.2: During minimax tree search, must

- A. Always (No: consider layer k , where mins of its children at layer $k+1$. If the $k+1$ already smaller than the current node, stop the minimization.)
- **B. Sometimes**
- C. Never (No: the event above may still happen)

Can We Do B

One **downside**: we had to exam

An idea to speed things up: **prun**

- Goal: want the same minimax valu
- We can get rid of bad branches
- Same principle as quiz question

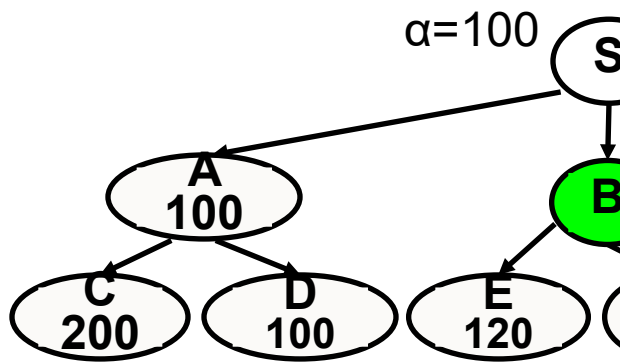
Minimax algorithm

max

min

max

min



Alpha-beta pr

function **Max-Value** (s, α, β)

inputs:

s : current state in game, Max about to play
 α : best score (highest) for Max along path to s
 β : best score (lowest) for Min along path to s

output: $\min(\beta, \text{best-score (for Max) available from } s)$

```
if (  $s$  is a terminal state )  
then return ( terminal value of  $s$  )  
else for each  $s'$  in Succ( $s$ )  
     $\alpha := \max(\alpha, \text{Min-value}(s', \alpha, \beta))$   
    if (  $\alpha \geq \beta$  ) then return  $\beta$  /* alpha pruning */  
return  $\alpha$ 
```

function **Min-Value**(s, α, β)

output: $\max(\alpha, \text{best-score (for Min) available from } s)$

```
if (  $s$  is a terminal state )  
then return ( terminal value of  $s$  )  
else for each  $s'$  in Succs( $s$ )  
     $\beta := \min(\beta, \text{Max-value}(s', \alpha, \beta))$   
    if (  $\alpha \geq \beta$  ) then return  $\alpha$  /* beta pruning */  
return  $\beta$ 
```

Alpha-Beta P

How effective is **alpha-beta**

- Depends on the order of success
 - Best case, the #of nodes to search is
 - Happens when each player's best move
 - The worst case is no pruning at all.
- In DeepBlue, the average branch factor with alpha-beta instead of 35-40

Minimax With H

Note that long games may require

- To deal with this: limit d for the search
- **Q:** What to do at depth d , but no terminal state?
 - **A:** Use a heuristic evaluation function

```
function MINIMAX( $x, d$ ) returns an estimate of  $x$ 's value
  inputs:  $x$ , current state in game
          $d$ , an upper bound on the search depth
  if  $x$  is a terminal state then return Max's payoff
  else if  $d = 0$  then return  $e(x)$ 
  else if it is Max's move at  $x$  then
    return max{MINIMAX( $y, d-1$ ) :  $y$  is a child of  $x$ }
  else return min{MINIMAX( $y, d-1$ ) :  $y$  is a child of  $x$ }
```

Heuristic Evaluation

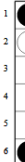
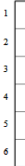
- $e(x)$ can be any computable function of features (like our linear models)

$$e(x) = w_1 f_1(x) + w_2 f_2(x)$$

- Chess example: $f_i(x) = \text{difference}$ between white and black, with i ranging over pieces
 - Set weights according to piece importance
 - E.g., $1(\# \text{ white pawns} - \# \text{ black pawns})$ (knights)

Going Further

- Monte Carlo tree search (MCTS)
 - Uses random sampling of the search space
 - Choose some children (heuristics to find promising ones)
 - Record results, use for future play
 - Self-play
- AlphaGo and other big results!



From Extensive Form back to

- A **pure strategy** for a player is the mapping between all possible states player can see, to the move the player would make.
- Player A has 4 pure strategies:
 - A's strategy I: $(1 \rightarrow L, 4 \rightarrow L)$
 - A's strategy II: $(1 \rightarrow L, 4 \rightarrow R)$
 - A's strategy III: $(1 \rightarrow R, 4 \rightarrow L)$
 - A's strategy IV: $(1 \rightarrow R, 4 \rightarrow R)$
- Player B has 3 pure strategies: $\begin{pmatrix} L \\ +7 \end{pmatrix}$
 - B's strategy I: $(2 \rightarrow L, 3 \rightarrow R)$
 - B's strategy II: $(2 \rightarrow M, 3 \rightarrow R)$
 - B's strategy III: $(2 \rightarrow R, 3 \rightarrow R)$
- How many pure strategies if each player can see N states, and has b moves at state?

Matrix Normal Form

A's strategy I: (1→L, 4→L)

A's strategy II: (1→L, 4→R)

A's strategy III: (1→R, 4→L)

A's strategy IV: (1→R, 4→R)

B's strategy I: (2→L, 3→R)

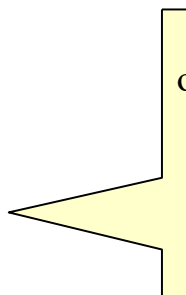
B's strategy II: (2→M, 3→R)

B's strategy III: (2→R, 3→R)

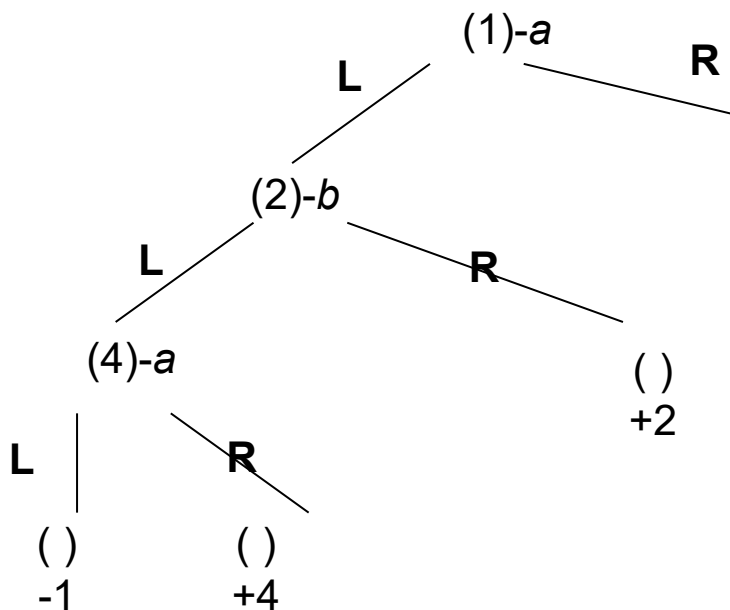
L
()
+7

The matrix normal form is the game value matrix indexed by each player's strategies.

	B-I	B-II	B-III
A-I	7	3	-1
A-II	7	3	4
A-III	5	5	5
A-IV	5	5	5

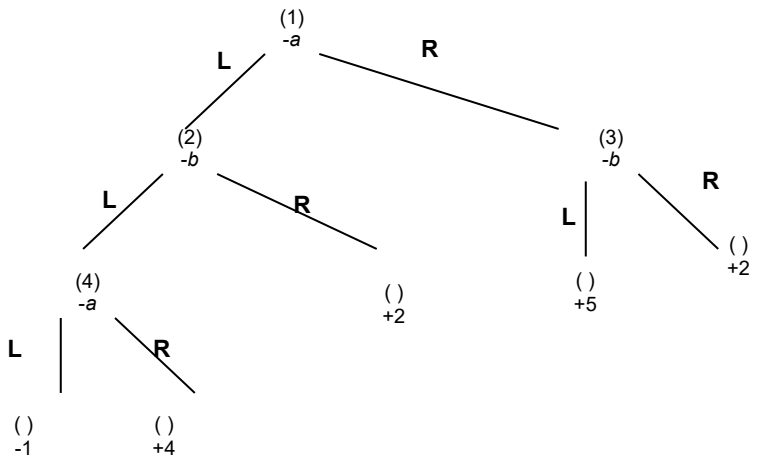


Another example of



- How many pure strategies
- How many does B have?
- What is the matrix form

Matrix normal form



- How many pure strategies does A have? 4
A-I (1→L, 4→L) A-II (1→L, 4→R) A-III (1→R, 4→L) A-IV (1→R, 4→R)
- How many does B have? 4
B-I (2→L, 3→L) B-II (2→L, 3→R) B-III (2→R, 3→L) B-IV (2→R, 3→R)
- What is the matrix form of this game?

Minimax in Matrix M

- Player A: for each strategy, consider all B's counter strategies (a row in the matrix), find the **minimum value** in that row. Pick the row with the maximum minimum value.
- Here $\text{maximin}=5$

L
()
+7



Minimax in Matrix No

- Player B: find the **maximum value** in each column. Pick the column with the minimum maximum value.
- Here minimax = 5

L
/
()
+7

Fundamental game theory result (proved by von Neumann):

*In a 2-player, zero-sum game of perfect information (sequential moves),
Minimax==Maximin. And there always exists an optimal pure strategy for each player.*



Minimax in Matrix No

Interestingly, A can tell B in advance what strategy A will use (the maximin), and this information will not help B!

Similarly B can tell A what strategy B will use.

In fact A knows what B's strategy will be.

And B knows A's too.

And A knows that B knows

...

The game is at an equilibrium

... a pure strategy for each player.

L
()
+7

ult

ame

Minimax in Matrix No

- We can also check for mutual best responses

L
/
()
+7

	B-I	B-II	B-III
A-I	7	3	-1
A-II	7	3	4
A-III	5	5	5
A-IV	5	5	5

Minimax in Matrix No

- We can also check for mutual best responses

L
/
()
+7

	B-I	B-II	B-III
A-I	7	3	-1
A-II	7	3	4
A-III	5	5	5
A-IV	5	5	5

Suggested R

Textbook: Artificial Intelligence: A Model
Stuart Russell and Peter Norvig. Pearson

- Chapters 5, 18